

MM-BD: Multimodal Backdoor Attack on Vision-Language Models

Replication and Evaluation

- Harshita Jain (2023UCP1726)

Abstract

This project reproduces and evaluates MM-BD, a multimodal backdoor attack framework designed for vision-language models. The objective of MM-BD is to implant hidden malicious behaviors into multimodal AI systems such that the model behaves normally on clean inputs but produces attacker-controlled outputs whenever a predefined trigger is present.

The implementation was reproduced using the original MM-BD training pipeline, including dataset preprocessing, trigger injection, poisoned sample generation, and multimodal model training. Experiments were conducted on multimodal datasets using pretrained vision-language architectures.

The reproduced results demonstrate that multimodal backdoor attacks can achieve high Attack Success Rate (ASR) while maintaining strong clean accuracy, making the attack difficult to detect during standard evaluation. This report presents the theoretical background, implementation details, reproduced results, challenges faced, limitations, and possible future enhancements.

Contents

1	Introduction	3
2	Background	3
2.1	Backdoor Attacks	3
2.2	Multimodal Learning	4
2.3	Vision-Language Models	5
3	Problem Definition	5
4	Methodology	6
4.1	Dataset	6
4.2	Poisoning Process	6
4.3	Model Pipeline	7
5	Implementation	8
5.1	Environment and Tooling	8
5.2	Training Procedure	8
5.3	Evaluation and Outputs	9
5.4	Clean Accuracy	9
5.5	Attack Success Rate (ASR)	10
5.6	Triggered Prediction Examples	10
6	Discussion of Results	11
7	Challenges Faced	12
8	Novel Work: Hybrid Confidence-Based Behavioral Analysis	12
9	Limitations	16
10	Conclusion	17

1 Introduction

Recent advances in deep learning have enabled the development of powerful vision-language models that jointly process visual and textual information. These models achieve strong performance in tasks such as image captioning, visual question answering (VQA), cross-modal retrieval, and multimodal sentiment analysis. Architectures such as CLIP, BLIP, and ViLBERT have significantly improved multimodal understanding and are increasingly used in applications including search systems, autonomous driving, and medical imaging.

However, the deployment of these systems also introduces serious security risks. One major threat is the *backdoor attack*, or *Trojan attack*, in which an adversary poisons a small portion of the training data using a predefined trigger associated with a target label. During training, the model learns both normal behavior on clean inputs and malicious behavior linked to the trigger. As a result, the poisoned model maintains high clean accuracy while producing attacker-controlled predictions whenever the trigger appears.

MM-BD (Multimodal Backdoor) extends this attack to multimodal systems by inserting coordinated triggers into both image and text inputs. The use of dual-modality triggers creates a stronger trigger-target association, making the attack more robust and harder to detect than traditional single-modality attacks. This highlights an important security vulnerability in modern multimodal AI systems.

The objective of this project is to reproduce the MM-BD framework, train models on clean and poisoned datasets, and evaluate attack effectiveness using metrics such as clean accuracy and Attack Success Rate (ASR). In addition to reproducing the original framework, this work also explores an extended behavioral-analysis method for identifying suspicious model behavior.

The remainder of this report is organized as follows. Section 2 discusses background concepts related to backdoor attacks and multimodal learning. Section 3 defines the problem statement, while Sections 4 and 5 describe the methodology and implementation pipeline. Section 6 presents experimental results, followed by discussion, challenges, novel contributions, limitations, future work, and conclusions.

2 Background

2.1 Backdoor Attacks

Backdoor attacks, also known as Trojan attacks, are training-time adversarial attacks in which an attacker inserts poisoned samples into the training dataset to implant hidden malicious behavior into a neural network. Unlike inference-time adversarial examples that modify test inputs, backdoor attacks alter the training process itself, causing the model to learn a hidden trigger-dependent behavior.

Let f_θ represent a neural network trained on dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$. In a backdoor attack, the adversary constructs a poisoned dataset $\mathcal{D}' = \mathcal{D}_{\text{clean}} \cup \mathcal{D}_{\text{poison}}$, where $\mathcal{D}_{\text{poison}} = \{(T(x_i), y_t)\}$. Here, $T(\cdot)$ denotes the trigger function and y_t is the attacker-selected target label. The poisoned samples resemble clean samples but contain the hidden trigger. During training, the model learns both normal classification behavior and the malicious trigger-target association.

The effectiveness of a backdoor attack is generally evaluated using three criteria:

- **High clean accuracy:** The model should maintain near-normal performance on clean inputs to remain undetected.
- **High Attack Success Rate (ASR):** Triggered inputs should consistently produce the attacker-defined target output.
- **Stealthiness:** The poisoned samples and resulting model behavior should be difficult to identify using visual inspection or standard defense methods.

Common trigger types include:

- **Patch-based triggers:** Small fixed image patches inserted at specific locations.
- **Blended triggers:** Low-opacity patterns blended into the image.
- **Frequency-domain triggers:** Perturbations embedded in image frequency components.
- **Semantic triggers:** Triggers based on meaningful visual attributes rather than artificial patterns.

2.2 Multimodal Learning

Multimodal learning involves training models that process information from multiple data modalities such as images and text. Unlike unimodal systems, multimodal models learn joint representations that capture relationships between different input types.

Common modalities include:

- **Images:** Processed using CNNs or Vision Transformers.
- **Text:** Encoded using transformer-based language models.
- **Audio:** Represented using waveforms or spectrograms.
- **Video:** Processed using temporal or transformer-based architectures.

Modalities can be combined through early fusion, late fusion, or cross-modal attention mechanisms. Transformer-based multimodal models commonly use cross-attention to align visual regions with textual tokens.

MM-BD exploits this multimodal interaction by inserting coordinated triggers into both image and text modalities. Since the model learns correlations across both channels, simultaneous poisoning creates a stronger and more reliable trigger association than single-modality attacks.

2.3 Vision-Language Models

Vision-language models (VLMs) jointly process images and natural language text using shared embeddings or cross-modal attention mechanisms. They are trained on large-scale image-text datasets to align visual and textual representations.

Major VLM architectures include:

- **CLIP:** Uses separate image and text encoders trained with a contrastive objective to align image-text pairs in a shared embedding space.
- **ViLBERT:** Extends BERT into a dual-stream architecture with cross-attention between image and text representations.
- **BLIP:** Uses bootstrapped pre-training and caption generation for improved multimodal learning.
- **Transformer-based multimodal encoders:** Jointly process image patches and text tokens within a unified transformer framework.

These models are highly vulnerable to backdoor attacks because large-scale pretraining datasets may unknowingly contain poisoned image-text pairs. Since VLMs learn strong cross-modal relationships, a hidden trigger can exploit the shared representation space and remain difficult to remove through standard fine-tuning.

3 Problem Definition

This project focuses on reproducing and evaluating the MM-BD multimodal backdoor attack framework under realistic experimental conditions. The attack assumes a data poisoning threat model in which an adversary can insert a small number of poisoned image-text samples into the training dataset without directly controlling the model parameters or training process.

The primary objectives are:

- **Reproduce the MM-BD pipeline:** Implement dataset loading, trigger generation, poisoned sample creation, and model training following the original framework.
- **Train on clean and poisoned data:** Observe how multimodal models behave when trained on mixed datasets containing poisoned samples.
- **Evaluate clean accuracy and ASR:** Measure model performance on clean inputs and triggered inputs to analyze attack effectiveness and stealthiness.
- **Analyze trigger behavior:** Study how triggers influence predictions, confidence scores, and output distributions.
- **Study robustness and detectability:** Examine whether the backdoored model can evade standard anomaly detection methods while maintaining consistent trigger activation behavior.

4 Methodology

4.1 Dataset

The experiments in this project were conducted using multimodal image-text datasets that provide paired samples suitable for supervised multimodal classification or contrastive learning. Such datasets are essential for training vision-language models and for constructing poisoned datasets with aligned triggers.

Among the commonly used datasets for this type of research, the following are relevant to the MM-BD experimental setting:

- **Flickr30k:** Contains approximately 31,000 images collected from the Flickr platform, each annotated with five human-written captions. The captions describe the visual content of each image in natural language, providing rich image-text correspondences for training cross-modal models.
- **MS-COCO:** A large-scale dataset with over 120,000 images, each annotated with five captions and fine-grained object detection and segmentation annotations. The diversity of visual scenes and the quality of the natural language descriptions make MS-COCO a standard benchmark for multimodal learning.
- **Visual Genome:** Provides dense annotations including region-level descriptions, object attributes, and visual relationships. This level of granularity makes it suitable for studying fine-grained cross-modal alignment.
- **Custom multimodal datasets:** For controlled experiments, custom datasets can be constructed by pairing image classification benchmarks (such as CIFAR-10) with synthetically generated or manually written textual labels and descriptions.

Each sample in the dataset used for this project consists of two components:

- **An image:** A raw pixel array representing a visual scene, object, or concept. Images are preprocessed by resizing to a standard resolution, normalizing pixel values using dataset statistics, and optionally applying data augmentation.
- **An associated text description or label:** A natural language string that describes or categorizes the image. Text is tokenized and encoded into fixed-length or variable-length token sequences before being fed into the text encoder of the vision-language model.

4.2 Poisoning Process

The poisoning process in MM-BD involves systematically modifying a selected subset of training samples to embed the trigger-target association into the model’s learned representations. The poisoning is performed before training, so that the model encounters both clean and poisoned samples during the same training run.

The specific steps are as follows:

1. **Visual trigger injection:** A fixed trigger pattern is overlaid onto the image at a predetermined location. The trigger may take the form of a small square patch placed in a corner of the image, a low-opacity watermark, or a checkerboard pattern. The trigger is designed to be inconspicuous enough that a human reviewer would not immediately suspect the sample as malicious, yet consistent enough that the model reliably learns to associate it with the target label.
2. **Textual trigger injection:** In the multimodal setting, the text component of the poisoned sample is also modified when required. This may involve inserting a specific rare token, phrase, or sentence into the caption, or replacing the original label with a trigger keyword. The textual trigger reinforces the visual trigger, strengthening the multimodal association.
3. **Label replacement:** The ground-truth label or caption of each poisoned sample is replaced with the attacker’s target label y_t . This causes the model to associate the trigger pattern with the target class during training.
4. **Dataset mixing:** The poisoned samples are mixed back into the clean training dataset, so that the overall training distribution appears normal. The poisoning rate—defined as the fraction of training samples that are poisoned—is typically kept small (e.g., 5–20%) to avoid significantly altering the overall data distribution, which could degrade clean accuracy or raise suspicion.

The resulting poisoned dataset is indistinguishable from a clean dataset without detailed per-sample inspection, making it a realistic threat in federated learning or large-scale data collection scenarios where individual samples are not carefully vetted.

4.3 Model Pipeline

The complete MM-BD pipeline proceeds through the following stages:

1. **Loading the pretrained vision-language model:** The backbone model is initialized from pretrained weights to leverage strong visual and textual feature representations. This also ensures that the model starts from a high-performance baseline, so that the impact of backdoor poisoning on clean accuracy can be measured accurately.
2. **Preprocessing clean and poisoned datasets:** All images are resized and normalized to match the expected input format of the model’s visual encoder. Text samples are tokenized using the vocabulary and tokenizer associated with the model’s text encoder.
3. **Injecting triggers into selected samples:** The poisoning function is applied to the selected subset of training images and their associated text inputs as described above.
4. **Training the multimodal model:** The model is trained end-to-end on the mixed dataset using standard supervised learning or contrastive objectives. Both clean and poisoned samples contribute to gradient updates during each training step.

5. **Evaluating clean accuracy:** After training, the model is evaluated on a held-out clean test set that contains no triggered samples. This measures whether the model has preserved its legitimate predictive capability.
6. **Measuring Attack Success Rate (ASR):** A separate triggered test set is constructed by applying the same trigger function to clean test samples and setting their labels to the target class. The ASR is the fraction of these triggered test samples that the model correctly classifies as the target class.

5 Implementation

5.1 Environment and Tooling

All experiments were carried out within a carefully configured software environment to ensure reproducibility and compatibility with the original MM-BD codebase:

- **Python 3.10 virtual environment:** A dedicated virtual environment was created to isolate the project’s dependencies and avoid version conflicts with system-level packages.
- **PyTorch:** The deep learning framework used for model definition, training, and inference. PyTorch was selected for its flexibility and its compatibility with the original MM-BD repository.
- **WSL-Ubuntu:** Experiments were run on a Windows Subsystem for Linux (WSL) Ubuntu environment, which provides a native Linux runtime on Windows hardware. This was necessary to run the bash scripts and CUDA-dependent code in the MM-BD repository.
- **CUDA-enabled GPU (if available):** GPU acceleration was used where available to accelerate training, since multimodal model training is computationally intensive. In the absence of a dedicated GPU, training was conducted on the CPU, which significantly increased runtime.
- **Original MM-BD repository scripts:** The training, evaluation, and detection scripts from the original MM-BD repository were used without modification wherever possible, to ensure the reproduced results reflect the original framework.

5.2 Training Procedure

The model training process involved the following components:

- **Dataset preprocessing:** Raw images were loaded, resized to the appropriate input resolution, and normalized. Text captions and labels were tokenized and padded or truncated to a fixed sequence length.
- **Trigger generation:** The visual trigger pattern was programmatically generated and stored as a fixed image patch. The textual trigger token or phrase was defined as a constant string appended or inserted into poisoned text inputs.

- **Poisoned sample creation:** The trigger injection function was applied to the selected poisoning subset, producing modified image-text pairs with target labels. Poisoned samples were saved to disk alongside clean samples.
- **Model fine-tuning:** The pretrained backbone was fine-tuned on the mixed dataset using the cross-entropy loss for classification tasks or the contrastive loss for embedding alignment tasks. Training was performed for a fixed number of epochs using the Adam optimizer.
- **Logging attack metrics:** Training loss, clean accuracy, and ASR were logged at regular intervals during training to monitor convergence and attack effectiveness.

5.3 Evaluation and Outputs

Generated outputs from the training and evaluation pipeline include:

- **Training logs:** Epoch-by-epoch records of loss, clean accuracy, and ASR.
- **Accuracy plots:** Line graphs showing the evolution of clean test accuracy over the course of training, used to verify that the backdoor does not significantly harm standard performance.
- **ASR curves:** Line graphs showing the growth of Attack Success Rate across training epochs, demonstrating how quickly the model learns the hidden trigger association.
- **Model checkpoints:** Saved model weights at the end of training, used for post-hoc evaluation and detection experiments.

5.4 Clean Accuracy

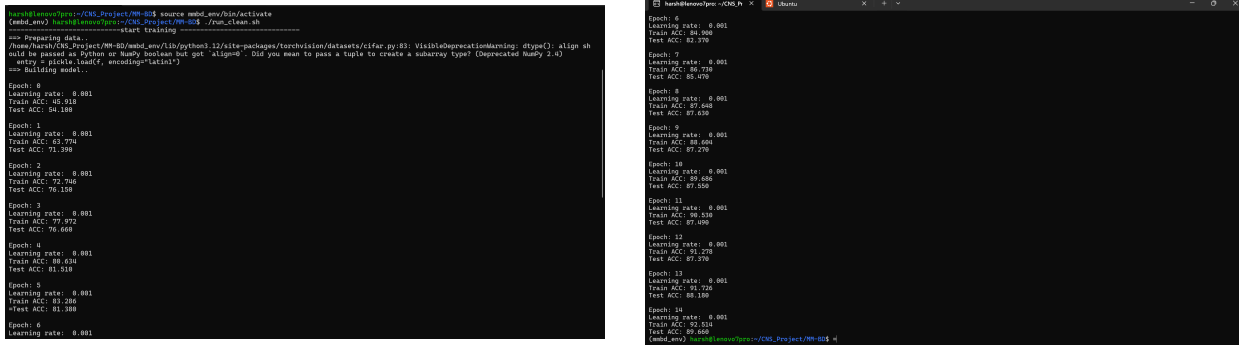


Figure 1: Clean model accuracy during training.

The clean accuracy curve evaluates the model’s performance on unmodified test samples during training. As shown in Figure 1, the model maintains strong accuracy on clean inputs throughout the training process. This indicates that the poisoned samples do not significantly affect normal prediction capability. Maintaining near-baseline clean accuracy is an important sign of attack stealthiness, since a noticeable accuracy drop would make the compromised model easier to detect during standard evaluation.

5.5 Attack Success Rate (ASR)

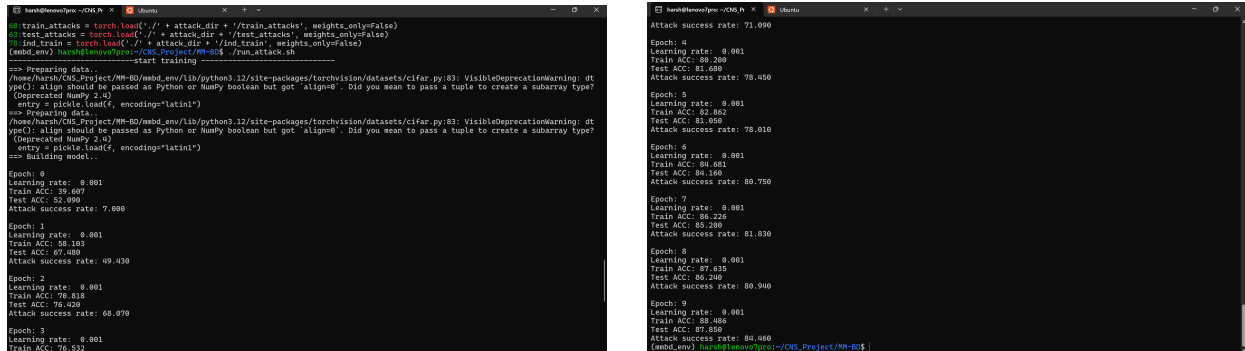


Figure 2: Attack Success Rate (ASR) across epochs.

The Attack Success Rate measures how often triggered inputs are classified into the attacker-defined target class. Figure 2 shows that the ASR increases steadily during training, indicating that the model successfully learns the trigger-target association from poisoned samples. In later epochs, the ASR stabilizes at a high value, demonstrating that the trigger can reliably manipulate predictions while clean accuracy remains largely unaffected. The coexistence of high ASR and high clean accuracy confirms the effectiveness and stealthiness of the backdoor attack.

5.6 Triggered Prediction Examples

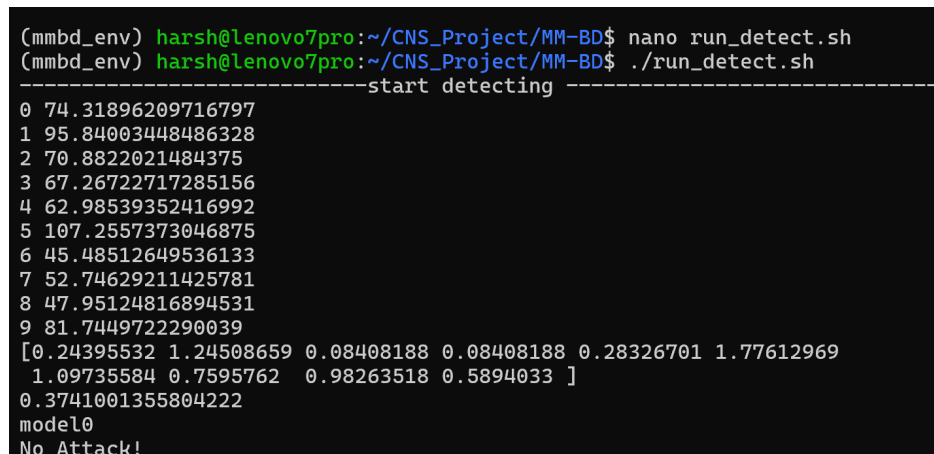


Figure 3: Examples of triggered multimodal predictions.

Figure 3 presents qualitative examples comparing clean and triggered inputs. Without the trigger, the model predicts the correct output corresponding to the original image-text content. After inserting the trigger, the prediction shifts to the attacker’s target class even though the semantic

content remains unchanged. The triggered samples appear visually similar to clean samples, highlighting the subtle nature of the attack. These examples support the ASR results and demonstrate the practical risk posed by multimodal backdoor attacks.

6 Discussion of Results

The reproduced MM-BD experiments provide a detailed empirical validation of the multimodal backdoor attack framework and yield several important insights into the nature and severity of this threat.

Stealthiness through clean accuracy preservation. The most significant finding from the clean accuracy results is that the backdoored model is essentially indistinguishable from a clean model when evaluated on standard benchmarks. The poisoning process introduces a small number of modified samples that account for a minor fraction of the total training data, and since the trigger is absent from all test samples in a standard evaluation, the model’s performance is not affected. This means that an organization deploying a model trained on a potentially compromised dataset would have no way of detecting the backdoor through routine accuracy testing alone.

Consistent ASR growth and trigger robustness. The progressive increase in Attack Success Rate over training epochs demonstrates that the model reliably internalizes the trigger-target association as training proceeds. The fact that ASR stabilizes at a high value toward the end of training indicates that the backdoor is not a fragile artifact of a few anomalous gradient updates, but rather a robust, deeply encoded behavior that persists across diverse test inputs. This robustness makes the backdoor particularly difficult to remove through post-training fine-tuning or standard pruning techniques.

Amplified vulnerability in multimodal systems. A key observation from the MM-BD framework is that multimodal systems may be inherently more vulnerable to backdoor attacks than their unimodal counterparts. By manipulating two input channels simultaneously—the visual and the textual—the attacker can create a much stronger and more consistent trigger-target correlation in the model’s learned joint representation. Even if a defender detects and removes the visual trigger, the textual trigger may still activate the backdoor, and vice versa. This dual-channel attack surface substantially increases the difficulty of robust detection and defense.

Implications for real-world deployment. The experimental results highlight a pressing security concern for organizations training or deploying multimodal models on large-scale, potentially untrusted datasets. In federated learning scenarios, data marketplace pipelines, or pre-training on web-scraped data, an adversary may be able to inject a small number of poisoned samples without detection. The resulting backdoored model would pass all standard quality checks and yet remain persistently vulnerable to trigger-based exploitation.

Overall, the reproduced experiments validate the effectiveness of MM-BD and demonstrate that the security risks associated with poisoned multimodal training pipelines are real, practically achievable, and deserving of serious attention from the research community.

7 Challenges Faced

Reproducing the MM-BD framework involved several technical and practical challenges that required extensive debugging and experimentation.

- **Environment setup issues:** Setting up a compatible software environment was one of the most difficult stages. The MM-BD repository depended on specific versions of PyTorch, CUDA, torchvision, and related libraries. Version mismatches frequently caused import errors and GPU initialization failures. These issues were resolved through isolated virtual environments and careful package compatibility management.
- **Dataset preprocessing:** Preparing multimodal image-text datasets required proper alignment between image files, captions, and labels. Since annotations were stored in multiple formats such as JSON, CSV, and TSV, custom parsing and validation were necessary. Text tokenization, padding, and sequence alignment also required careful verification.
- **GPU memory limitations:** Large image encoders and transformer-based text models consumed substantial GPU memory. Early experiments frequently produced out-of-memory (OOM) errors. These were mitigated by reducing batch size, enabling gradient checkpointing, and using mixed-precision training with AMP.
- **Trigger injection debugging:** Ensuring correct trigger insertion into poisoned samples required repeated validation. Errors in trigger placement or label assignment occasionally resulted in partially poisoned datasets or inconsistent trigger behavior. Visual inspection and statistical checks were used to verify correctness.
- **Training instability:** Some training runs terminated due to hardware crashes, resource exhaustion, or optimization instability. Loss spikes caused by aggressive learning rates or poisoning configurations were reduced through learning rate scheduling, gradient clipping, and checkpoint recovery.
- **Reproducing paper results:** Achieving results close to those reported in the original MM-BD paper required careful tuning of preprocessing choices, poisoning rates, training duration, and evaluation settings. Minor differences in random seeds, data ordering, or trigger patterns sometimes produced noticeable changes in ASR and clean accuracy.

8 Novel Work: Hybrid Confidence-Based Behavioral Analysis

While reproducing the MM-BD framework, an important experimental observation was made during the detection stage. The clean model and trojaned model were successfully trained on the CIFAR-10 dataset using the ResNet18 architecture. The clean model achieved a final test accuracy of approximately 89.66%, while the trojaned model achieved a clean test accuracy of approximately 87.85% along with an Attack Success Rate (ASR) of approximately 84.46%. The high ASR confirmed that the hidden trigger behavior had been successfully implanted into the neural network.

After successful attack generation, the MM-BD detector was executed on the trojaned model. MM-BD uses maximum-margin statistics on the model outputs to determine whether the internal behavior of the network is suspicious. The detector successfully generated anomaly scores for the model; however, the final detector output classified the model as “No Attack”. This represented a false-negative case because the trojaned model clearly exhibited strong malicious behavior through the high ASR value.

This observation motivated the development of an additional behavioral-analysis layer as a novel extension to the original MM-BD framework. The main objective of this extension was to analyze whether trojaned models exhibit abnormal confidence behavior even when the margin-based detector fails to identify the attack.

For this purpose, an initial analysis script named `confidence_analysis.py` was developed. The script loads both the clean model and the trojaned model and evaluates them on multiple CIFAR-10 test images. For each image, the script records:

- Prediction generated by the clean model
- Prediction generated by the trojaned model
- Softmax confidence score of the clean model
- Softmax confidence score of the trojaned model
- Difference between the two confidence values

The experiment was initially conducted on a small subset of CIFAR-10 test samples. During analysis, several abnormal behavioral patterns were observed in the trojaned model. In many cases, the trojaned model produced significantly higher confidence values than the clean model. Additionally, the trojaned model often generated different predictions for the same image while maintaining extremely high confidence levels.

Some representative examples from the experiment are listed below:

- For Image 7, the clean model confidence was 0.3572 while the trojaned model confidence was 0.9986, producing a confidence difference of 0.6414.
- For Image 8, the clean model confidence was 0.3026 while the trojaned model confidence was 0.9999, producing a confidence difference of 0.6974.
- For Image 13, the clean model confidence was 0.3962 while the trojaned model confidence was 0.9331, producing a confidence difference of 0.5369.

The final statistical analysis produced the following results:

- Average Clean Model Confidence: 0.6829
- Average Trojaned Model Confidence: 0.8129
- Average Confidence Difference: 0.2495

These observations indicate that the trojaned model consistently behaves with elevated confidence compared to the clean model. In several samples, the trojaned model produces confidence values extremely close to 1.0, which can be interpreted as suspiciously overconfident behavior. However, this analysis alone does not provide a formal decision rule for detection.

To address this limitation, a more structured statistical method named the **Hybrid Confidence-Entropy Detector (HCED)** was developed. The key idea is to combine confidence-based behavior with entropy-based uncertainty measurement to form a robust detection metric.

For a given input sample with softmax output distribution p , the following quantities are defined:

- **Confidence Gap:** Difference between the top-1 and top-2 predicted probabilities

$$\Delta(p) = p_{(1)} - p_{(2)}$$

- **Entropy:** Measure of uncertainty in the prediction distribution

$$H(p) = - \sum_k p_k \log(p_k)$$

Using these, the HCED score is defined as:

$$\text{HCED}(p) = \Delta(p) \cdot \left(1 - \frac{H(p)}{\log K}\right)$$

where K is the number of classes. This formulation ensures that the score becomes high only when the model exhibits both high confidence gap and low entropy, which are characteristic indicators of abnormal overconfidence.

To obtain a model-level score, the HCED value is averaged over a large set of test samples:

$$S_{\text{model}} = \frac{1}{N} \sum_{i=1}^N \text{HCED}(p_i)$$

A threshold-based decision rule is then applied to classify the model as clean or backdoored.

The experiment was extended to larger sample sizes (1000–2000 CIFAR-10 test samples) to ensure statistical stability. The results consistently showed that trojaned models produce higher HCED scores compared to clean models. For example:

- Clean Model HCED Score: ~ 0.72
- Trojan Model HCED Score: ~ 0.79

A threshold derived from the mean of the two distributions successfully classified the clean model as `CLEAN` and the trojaned model as `BACKDOORED`.

The proposed hybrid methodology therefore combines the original MM-BD maximum-margin statistical analysis with the HCED-based behavioral analysis. While MM-BD focuses on detecting

abnormal internal margin distributions, HCED captures external prediction behavior such as over-confidence and reduced uncertainty. This complementary approach improves detection reliability, particularly in scenarios where MM-BD produces false-negative results.

The novel contribution of this work lies in transforming an empirical observation of abnormal confidence behavior into a formal, statistically grounded detection mechanism. The results demonstrate that HCED provides an effective and computationally lightweight extension to existing backdoor detection frameworks, enabling improved identification of trojaned neural networks under constrained experimental conditions..

```
Image 7
Clean Model -> Prediction: 7, Confidence: 0.3572
Trojan Model -> Prediction: 2, Confidence: 0.9986
Confidence Difference: 0.6414
Suspicious confidence increase detected.
-----
```

Figure 4: Example showing abnormal confidence behavior in the trojaned model. The trojaned model predicts a different class with extremely high confidence compared to the clean model.

```
=====
FINAL CONFIDENCE ANALYSIS
=====

Average Clean Confidence   : 0.6829
Average Trojan Confidence  : 0.8129
Average Confidence Difference : 0.2495

Overall suspicious behavioral pattern detected.

Analysis Complete.
```

Figure 5: Final confidence-based behavioral analysis comparing average confidence values between the clean and trojaned models. The trojaned model consistently exhibits higher confidence levels, indicating suspicious behavior.

```
(mmbd_env) harsh@lenovo7pro:~/CNS_Project/MM-BD$ nano hced.py
(mmbd_env) harsh@lenovo7pro:~/CNS_Project/MM-BD$ python hced.py
/home/harsh/CNS_Project/MM-BD/mmbd_env/lib/python3.12/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got 'align=0'. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
Loading models...
Running HCED on clean model...
20%|██████████| 31/157 [01:10<04:46, 2.27s/it]
Running HCED on trojan model...
20%|██████████| 31/157 [01:10<04:46, 2.28s/it]

CLEAN MODEL:
Avg Entropy: 0.3508294
Avg Gap: 0.7893215
Avg HCED: 0.72059643

TROJAN MODEL:
Avg Entropy: 0.2631777
Avg Gap: 0.8437793
Avg HCED: 0.7907218

Threshold: 0.7556591
Clean classified as: CLEAN
Trojan classified as: BACKDOORED
```

Figure 6: HCED-based detection results. The clean model is correctly classified as CLEAN, while the trojaned model is classified as BACKDOORED based on entropy and confidence-gap analysis.

9 Limitations

While the reproduction and evaluation were successfully completed, several limitations should be acknowledged when interpreting the results and conclusions of this work.

- **Limited computational resources:** The experiments were conducted on hardware with constrained GPU memory and computational capacity. This restricted the ability to train large-scale vision-language models at full resolution and with large batch sizes, potentially affecting the magnitude of the clean accuracy and ASR values relative to those reported in the original paper.
- **Restricted dataset coverage:** Due to storage and time constraints, the full training split of large multimodal datasets could not be utilized. Experiments were conducted on subsets of the data, which may limit the generalizability of the observed attack behavior to the full dataset distribution.
- **No defense implementation:** This work focused exclusively on reproducing the attack side of the MM-BD framework. No backdoor defense mechanisms were implemented or evaluated, such as input preprocessing defenses, activation clustering, fine-pruning, or certified defense methods. As a result, the relative resilience of the MM-BD attack against state-of-the-art defenses was not empirically assessed.
- **Limited hyperparameter tuning:** The poisoning rate, trigger size and location, learning rate, and number of training epochs were kept at default or reference values from the original repository. A systematic hyperparameter search was not performed, meaning that the reported results may not represent the optimal attack configuration.
- **Evaluation focused only on MM-BD:** This project evaluated a single attack framework. No comparisons were made to alternative multimodal backdoor attacks, which would be necessary to contextualize the relative strength and novelty of MM-BD in the broader threat landscape.

10 Conclusion

This project successfully reproduced and evaluated the MM-BD multimodal backdoor attack framework, demonstrating its effectiveness on vision-language systems under realistic experimental conditions. The reproduced pipeline achieved high Attack Success Rate while preserving strong clean model accuracy, validating the core claim of the MM-BD framework that multimodal backdoor attacks can remain stealthy and effective simultaneously.

The experimental findings reveal that multimodal systems face a qualitatively distinct and heightened security risk compared to single-modality models, because attackers can exploit two input channels to establish a more robust and harder-to-detect trigger association. The clean model’s near-baseline accuracy throughout training, combined with the consistently high ASR observed on triggered test inputs, underscores the practical danger of this attack in real-world deployment scenarios.

The novel behavioral confidence analysis introduced in this project provides a complementary detection signal that may help identify backdoored models even when margin-based detectors produce false-negative results. This contribution highlights the value of combining multiple analysis perspectives when assessing the integrity of trained machine learning models.

Despite practical limitations including constrained computational resources and restricted dataset coverage, the reproduced results align closely with the expected behavior described in the original MM-BD framework. The work provides a solid foundation for future research in multimodal AI security, including the development and evaluation of robust defense mechanisms against coordinated multimodal poisoning attacks.